I2C Protocol

Basic Architecture

- One Master controls communication
- Multiple Slaves respond to master

The master: - Generates clock (SCL) - Starts communication - Sends address and data

The slave: - Listens to address - Responds with ACK - Receives or sends data

Electrical Behavior

I2C uses **open-drain logic**, which is different from normal digital logic.

Normal Digital Logic

- 0 driven LOW
- 1 driven HIGH

I2C Logic

- 0 actively driven LOW
- 1 NOT driven (line becomes HIGH due to pull-up resistor)

Pull-Up Behavior

Both SDA and SCL lines are connected to VDD through resistors.

This means:

- If no device drives the line, it becomes HIGH
- If any device pulls it LOW it becomes LOW

Rule

SDA must change only when SCL is LOW.

If SDA changes when SCL is HIGH:

- HIGH LOW START condition
- LOW HIGH STOP condition

I2C Transaction

Example:

- Address = 0x50
- Data = 0xA5, 0x5A

Step-by-Step Flow

1. Idle State

SDA = HIGH SCL = HIGH

2. START Condition

SDA goes from HIGH to LOW while SCL is HIGH This signals beginning of communication.

3. Address Transmission

Address = 0x50 Binary = 1010000 Add

write bit (0):

10100000

This is sent bit-by-bit on SDA, synchronized with SCL.

4. ACK (Acknowledgment)

After 8 bits:

- Master releases SDA
- Slave pulls SDA LOW

ACK = 0 (LOW)

5. Data Transmission (First Byte) Data = 0xA5 = 10100101 Sent

bit-by-bit.

6. ACK Again

Slave pulls SDA LOW again.

7. Data Transmission (Second Byte)

Data = 0x5A = 01011010

8. STOP Condition SDA goes LOW to

HIGH while SCL is HIGH

This ends communication.

Flow Summary

START - ADDRESS - ACK - DATA - ACK - DATA - ACK - STOP

Code Logic (Open-Drain Implementation):

```
assign sda = (mst_sda_oe || slv_sda_oe) ? 1 b0 : 1 bz;
```

1'bz:

- Z = High Impedance
- Means "not driving the wire"

Step 2: Why Z becomes HIGH?

Because of:

```
tri1 sda;
```

This means:

- Default value = HIGH
- Due to pull-up resistor

Working of the Code

Case 1: No one drives SDA

mst_sda_oe = 0 slv_sda_oe = 0

(0||0) = 0 : sda=z

Result:

sda = Z - becomes HIGH

Case 2: Master drives LOW mst_sda_oe = 1

(1||0) = 1 :

sda=0 Result:

sda=0

Case 3: Slave drives LOW (ACK)

(0||1) = 1 : sda=0

slv_sda_oe = 1

Result: sda =

0

Case 4: Both drive

(1||1) = 1 : sda=0 : Result still LOW (no conflict)

Truth Table

Master	Slave	SDA
0	0	HIGH
1	0	LOW
0	1	LOW
1	1	LOW

This is called **wired-AND behavior**:

- LOW dominates
- HIGH is passive

Design Implemented (I2C Master)

The design implemented is an I2C master controller using a Finite State Machine (FSM).

FSM States

- IDLE
- START
- SEND_ADDRESS
- WAIT_ACK
- SEND_DATA
- WAIT_ACK
- STOP

Internal Components

- State register
- Bit counter
- Multiplexers
- Control logic
- Tri-state SDA control

Working

- FSM transitions through states
- Generates START and STOP
- Sends address and data
- Waits for ACK from slave

Interface (i2c_if.sv)

The interface connects the DUT and UVM testbench.

Key Signals

- scl (clock line)
- sda (data line)
- mst_scl_oe (master clock control)
- mst_sda_oe (master data control)
- slv_sda_oe (slave ACK control)

Why This is Needed

In I2C:

- Multiple devices share same line
- No signal conflict allowed Using open-drain ensures: Safe communication, No short circuit

- Devices never drive HIGH
- They only pull LOW
- HIGH is created automatically